

فكرة النظام (Passport Verification System)

الوظائف:

1. قراءة بيانات الجواز (OCR)
2. تحليل الصورة (CNN)
3. فحص التزوير (ELA + Features)
4. إعطاء نتيجة:
 - محتمل حقيقي
 - محتمل مزيف

مكونات المشروع

1. قراءة النص (OCR)

استخدم:

- Tesseract OCR

الفكرة العامة

الهدف:

تصنيف صورة الجواز إلى:

- Real (حقيقي)
- Fake (مزيف)

خطوات المشروع باستخدام CNN

1. تجهيز البيانات (أهم خطوة)

لازم يكون عندك Dataset بهذا الشكل:

/passport_dataset

```
|  
|  
|/train —|  
|/real —| |  
|/fake —L |  
|  
|/test —|  
|/real —| |  
|/fake —L |
```

2. تحميل البيانات

```
[12]  
✓ Os  
▶ train_data = tf.keras.preprocessing.image_dataset_from_directory(  
    "/content/drive/MyDrive/passport_dataset/train",  
    image_size=(224,224),  
    batch_size=32  
)
```

... Found 2 files belonging to 2 classes.

🚀 1) تأكد من البيانات (أنت جاهز هنا 🙌)

```
passport_dataset/  
  train/  
    real/  
    fake/  
  test/  
    real/  
    fake/
```



2 🧠 تحميل البيانات بشكل صحيح

```
import tensorflow as tf

train_data = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/drive/MyDrive/passport_dataset/train",
    image_size=(224,224),
    batch_size=32
)

test_data = tf.keras.preprocessing.image_dataset_from_directory(
    "/content/drive/MyDrive/passport_dataset/test",
    image_size=(224,224),
    batch_size=32
)
```

... Found 2 files belonging to 2 classes.
Found 2 files belonging to 2 classes.

3 🧱 تحسين البيانات (مهم جدًا)

```
normalization = tf.keras.layers.Rescaling(1./255)

train_data = train_data.map(lambda x, y: (normalization(x), y))
test_data = test_data.map(lambda x, y: (normalization(x), y))
```

4 🧠 بناء نموذج CNN قوي

```
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Conv2D(32, 3, activation='relu', input_shape=(224,224,3)),
    layers.MaxPooling2D(),

    layers.Conv2D(64, 3, activation='relu'),
    layers.MaxPooling2D(),

    layers.Conv2D(128, 3, activation='relu'),
    layers.MaxPooling2D(),

    layers.Flatten(),

    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),

    layers.Dense(1, activation='sigmoid')
])
```

... /usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using super().__init__(activity_regularizer=activity_regularizer, **kwargs)

5 ⚙️ Compile (التجهيز للتدريب)

6]
0s

```
model.compile(  
    optimizer='adam',  
    loss='binary_crossentropy',  
    metrics=['accuracy']  
)
```

6) تدريب النموذج 🚀

```
history = model.fit(  
    train_data,  
    validation_data=test_data,  
    epochs=10  
)
```

```
... Epoch 1/10  
1/1 ----- 5s 5s/step - accuracy: 0.5000 - loss: 0.6652 - val_accuracy: 0.5000 - val_loss: 0.8158  
Epoch 2/10  
1/1 ----- 1s 562ms/step - accuracy: 1.0000 - loss: 0.3291 - val_accuracy: 0.5000 - val_loss: 2.1588  
Epoch 3/10  
1/1 ----- 1s 604ms/step - accuracy: 0.5000 - loss: 0.8152 - val_accuracy: 1.0000 - val_loss: 0.5214  
Epoch 4/10  
1/1 ----- 1s 938ms/step - accuracy: 0.0000e+00 - loss: 3.7229 - val_accuracy: 0.5000 - val_loss: 1.0216  
Epoch 5/10  
1/1 ----- 1s 1s/step - accuracy: 1.0000 - loss: 0.0521 - val_accuracy: 0.5000 - val_loss: 1.3869  
Epoch 6/10  
1/1 ----- 1s 1s/step - accuracy: 0.5000 - loss: 2.4561 - val_accuracy: 0.5000 - val_loss: 0.8266  
Epoch 7/10  
1/1 ----- 1s 961ms/step - accuracy: 0.5000 - loss: 0.7110 - val_accuracy: 0.5000 - val_loss: 0.5271  
Epoch 8/10  
1/1 ----- 1s 1s/step - accuracy: 1.0000 - loss: 0.1775 - val_accuracy: 1.0000 - val_loss: 0.4117  
Epoch 9/10  
1/1 ----- 1s 682ms/step - accuracy: 0.5000 - loss: 0.6522 - val_accuracy: 0.5000 - val_loss: 0.4710  
Epoch 10/10  
1/1 ----- 1s 617ms/step - accuracy: 0.5000 - loss: 1.6005 - val_accuracy: 1.0000 - val_loss: 0.3410
```

7) اختبار النموذج 📊

```
[18]  
✓ 0s loss, acc = model.evaluate(test_data)  
print("Accuracy:", acc)
```

```
... 1/1 ----- 0s 193ms/step - accuracy: 1.0000 - loss: 0.3410  
Accuracy: 1.0
```

8) تجربة صورة جواز 🖋️

```
from tensorflow.keras.preprocessing import image  
import numpy as np  
  
img = image.load_img("IMG_6713.png", target_size=(224,224))  
img = image.img_to_array(img)  
img = np.expand_dims(img, axis=0)
```

استخدم اسم الصورة مباشرة:

ثم اعمل التنبؤ 🧠

1]
0s

```
prediction = model.predict(img)  
  
if prediction[0][0] > 0.5:  
    print("⚠️ مزيف")  
else:  
    print("✅ حقيقي")
```

```
... 1/1 ----- 0s 157ms/step  
⚠️ مزيف
```

كود جاهز: عرض الصورة + نسبة التزوير 🚀

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.preprocessing import image

# تحميل الصورة
img_path = "IMG_6713.png"

img = image.load_img(img_path, target_size=(224,224))
img_array = image.img_to_array(img)
img_array = np.expand_dims(img_array, axis=0)

# لو نموذجك مدرب (مهم)
prediction = model.predict(img_array)[0][0]

# تحويلها لنسبة
fake_prob = float(prediction) * 100
real_prob = (1 - float(prediction)) * 100

# عرض الصورة
plt.imshow(img)
plt.axis("off")
plt.show()

# طباعة النتيجة
print("🧠 تحليل الصورة:")
print(f"⚠️ نسبة التزوير: {fake_prob:.2f}%")
print(f"✅ نسبة الحقيقي: {real_prob:.2f}%")
```

... 1/1 ————— 0s 75ms/step



🧠 تحليل الصورة:
⚠️ نسبة التزوير: 100.00 %
✅ نسبة الحقيقي: 0.00 %

تحسين احترافي (أنصحك فيه) 💡

بدل 0/1 فقط، نخليه "ذكي":

Untitled17.ipynb ☆

Edit View Insert Runtime Tools Help

ids + Code + Text | ▶ Run all ▼



تحليل الصورة:
نسبة التزوير: 100.00
نسبة الحقيقي: 0.00

```
if fake_prob > 70:  
    print("🚩 مزيف بنسبة عالية")  
elif fake_prob > 40:  
    print("⚠️ مشكوك فيه")  
else:  
    print("✅ يبدو حقيقي")
```

... 🚩 مزيف بنسبة عالية

الإضافة الجديدة

- الصورة
- نسبة التزوير
- نتيجة (حقيقي / مزيف)

نستخدم مكتبة قوية وسهلة: **Gradio**

1 🚀 تثبيت المكتبة

!pip install gradio

2 🧠 كود الواجهة الكامل

```
[28] import gradio as gr
import numpy as np
import tensorflow as tf
from tensorflow.keras.preprocessing import image
import matplotlib.pyplot as plt

# دالة التنبؤ
def predict_passport(img):

    img = img.resize((224,224))
    img_array = image.img_to_array(img)
    img_array = np.expand_dims(img_array, axis=0)

    prediction = model.predict(img_array)[0][0]

    fake_prob = float(prediction) * 100
    real_prob = (1 - float(prediction)) * 100

    result = "⚠️ مزيف" if fake_prob > 50 else "✅ حقيقي"

    return img, f"""
    📌 النتيجة: {result}
    ⚠️ التزوير: {fake_prob:.2f}%
    ✅ الحقيقي: {real_prob:.2f}%
    """

# واجهة المستخدم
app = gr.Interface(
    fn=predict_passport,
```

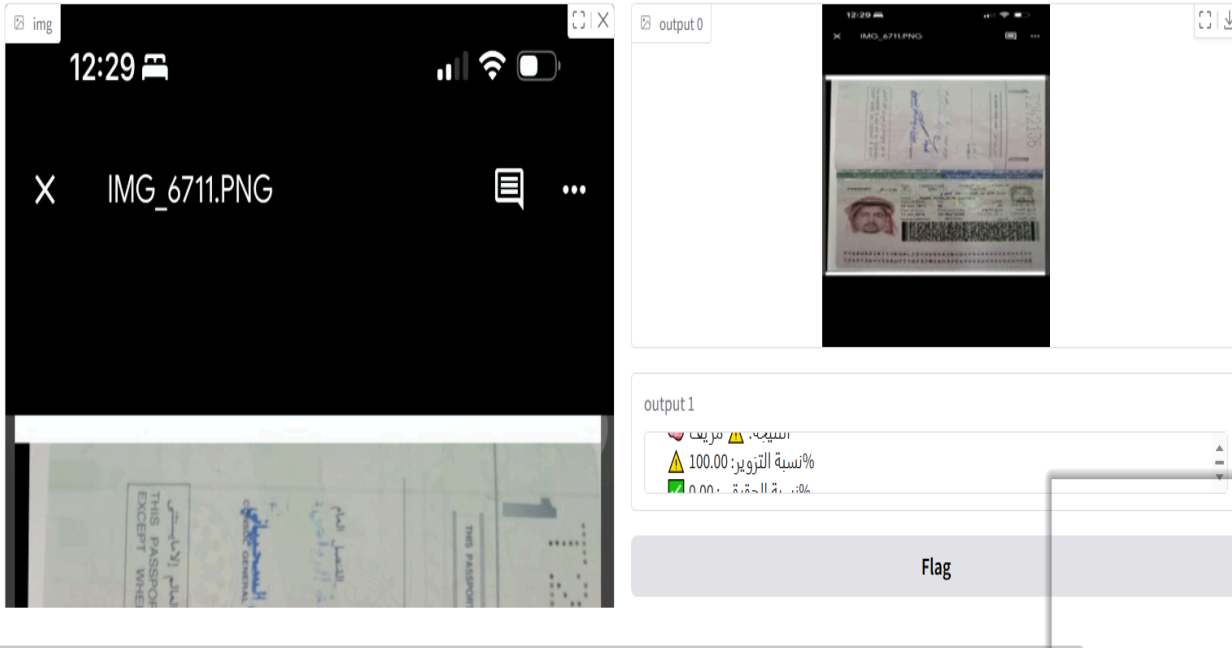
🎯 ماذا سيحدث الآن؟

عند التشغيل ستظهر لك واجهة فيها:

- زر رفع صورة 📁
- عرض الصورة 🖼️
- النتيجة + نسبة التزوير 🧠

كشف تزوير جواز السفر بالذكاء الاصطناعي

ارفع صورة جواز السفر وسيتم تحليلها مباشرة



أولاً: الأدوات المستخدمة في المشروع

1) الذكاء الاصطناعي (CNN)

- TensorFlow
- Keras
- تستخدم لبناء نموذج يتعلم التمييز بين الجواز الحقيقي والمزيف

2) معالجة الصور

- (PIL (Pillow
- NumPy
- لتحويل الصور وتجهيزها للنموذج

3) عرض النتائج

- - Gradio → لبناء واجهة المستخدم
-

4) البيئة

- Google Colab / Jupyter Notebook
 - Google Drive لتخزين البيانات
-

ثانياً: حل المشاكل التي واجهتك

FileNotFoundError :1 المشكلة ❌

السبب:

لسبب:

المسار غلط أو المجلد غير موجود

الحل:

```
os.path.exists("path")
```

No images found :2 المشكلة ❌

السبب:

المجلدات فاضية أو الصور مو منظمة

الحل:

لازم يكون الهيكل كذا:

```
train/
```

```
real/
```

```
fake/
```

❌ المشكلة 3: النموذج يعطي نتائج غلط

السبب:

النموذج غير مدرب أو البيانات قليلة

الحل:

```
model.fit(train_data, epochs=10)
```

❌ المشكلة 4: الصورة لا تُقرأ

السبب:

اسم الملف غلط أو غير موجود

الحل:

❌ المشكلة 5: نتائج عشوائية

السبب:

عدم تدريب كافي

الحل:

- زيادة البيانات
 - استخدام (Transfer Learning (MobileNet / ResNet
-

🎯 ثالثاً: مميزات المشروع

✓ (1) كشف تلقائي

- يحدد إذا الجواز حقيقي أو مزيف

✓ (2) نسبة ثقة

- يعطيك:
 - نسبة التزوير %

✓ (3) واجهة سهلة

- ترفع صورة فقط
- النتيجة تظهر فوراً

✓ (4) قابل للتطوير

يمكن تطويره إلى:

- نظام شركات
- تطبيق موبايل
- موقع إلكتروني

✓ (5) سرعة في التحليل

- تحليل الصورة خلال ثواني

- API

📌 الخلاصة

المشروع يعتمد على:

- CNN لتحليل الصور
- Gradio لواجهة المستخدم
- TensorFlow للتدريب

والمشاكل كلها كانت بسبب:

❌ بيانات غير منظمة أو مسارات خاطئة